Question 1 À partir des exemples d'entraînement ci-dessous, utilisez scikit-learn pour implémenter un classifieur perceptron1 qui classe les étudiants en deux catégories, en prédisant qui obtiendra un « A » cette année à partir d'un vecteur de caractéristiques d'entrée X. Voici de nouveau les données d'entraînement :

	Caractéristique (x)				Sortie f(x)
Étudiant	« A » l'année dernière ?	Cheveux noirs ?	Travaille dur ?	Boit ?	« A » cette année ?
X1: Richard	Oui	Oui	Non	Oui	Non
X2: Alan	Oui	Oui	Oui	Non	Oui
X3: Alison	Non	Non	Oui	Non	Non
X4: Jeff	Non	Oui	Non	Oui	Non
X5: Gail	Oui	Non	Oui	Oui	Oui
X6: Simon	Non	Oui	Oui	Oui	Non

Utilisez les imports Python suivants pour le perceptron:

```
import numpy as np
from sklearn.linear_model import Perceptron
```

Toutes les caractéristiques doivent être numériques pour entraîner le classifieur. Vous devez donc convertir les valeurs de caractéristiques « Oui » et « Non » en leur représentation binaire :

```
dataset = [1,1,0,1,0],
          [1,1,1,0,1],
          [0,0,1,0,0],
          [0,1,0,1,0],
          [1,0,1,1,1],
          [0,1,1,1,0]
```

Pour nos vecteurs de caractéristiques, nous avons besoin des quatre premières colonnes :

```
X = les 4 premières caractéristiques
```

et, pour les étiquettes d'entraînement, nous utilisons la dernière colonne du jeu de données :

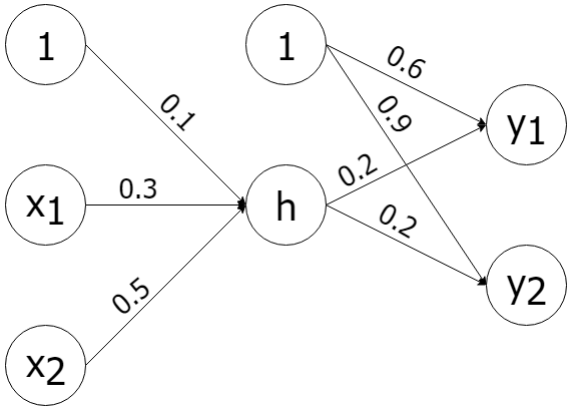
```
y = la dernière colonne
```

[1] https://scikit-learn.org/stable/modules/linear_model.html#perceptron

(a) Créez maintenant un classifieur perceptron et entraînez-le.

(b) Appliquez l'entraînement à tous les échantillons d'entraînement et affichez la prédiction.

Question 2 Considérez le réseau neuronal présenté ci-dessous. Il se compose de 2 nœuds d'entrée, de 1 nœud caché et de 2 nœuds de sortie, avec un biais supplémentaire dans la couche d'entrée et un biais dans la couche cachée. Tous les nœuds des couches cachées et de sortie utilisent la fonction d'activation sigmoïde (σ).



(a) Calculez la sortie de y1 et y2 lorsque le réseau reçoit x = (1, 0) en entrée.

(b) Supposez que la sortie attendue pour l'entrée x = (1, 0) soit t = (0, 1). Calculez les poids mis à jour après la rétropropagation de l'erreur pour cet échantillon. Supposez que le taux d'apprentissage η = 0,1.

Question 3 Voyons comment construire des réseaux neuronaux multicouches à l'aide de scikit-learn.2

(a) Implémentez l'architecture de la question précédente à l'aide de scikit-learn et utilisez-la pour apprendre la fonction XOR, qui n'est pas linéairement séparable. Utilisez les imports Python suivants :

```
import numpy as np
from sklearn.neural network import MLPClassifier
```

Voici les données d'entraînement pour la fonction XOR

```
dataset = [1,1,0],
          [0,1,1],
          [1,0,1],
          [0,0,0]
```

Pour nos vecteurs de caractéristiques, nous avons besoin des deux premières colonnes :

```
X = les deux premières colonnes
```

et, pour les étiquettes d'entraînement, nous utilisons la dernière colonne du jeu de données :

```
y = la dernière colonne
```

Vous pouvez maintenant créer un perceptron multicouche à l'aide du classifieur MLP de scikit-learn3. Vous pouvez choisir de nombreux paramètres à définir et à personnaliser ; ici, vous devez définir hidden_layer_sizes. Pour ce paramètre, transmettez un tuple constitué du nombre de neurones souhaité dans chaque couche, où la nième entrée du tuple représente le nombre de neurones dans la nième couche du modèle MLP. Vous devez également définir l'activation sur « logistic », qui correspond à la fonction sigmoïde logistique. Les détails relatifs aux biais et aux poids sont définis implicitement dans la définition de la fonction.

(b) Appliquez maintenant le modèle entraîné à tous les échantillons d'entraînement et affichez sa prédiction.

Comme vous pouvez le constater, notre unique couche cachée comportant un seul neurone n'apprend pas correctement la fonction XOR. Il est toujours utile d'expérimenter différentes configurations de réseau.

[2] https://scikit-learn.org/stable/modules/neural_networks_supervised.html
[3] https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Question 4 Considérez l'architecture de réseau neuronal ci-dessous et utilisez-la pour classifier le jeu de données des fleurs Iris.

(a) Chargez Iris à partir des jeux de données intégrés de scikit-learn4. Comme précédemment, utilisez la fonction auxiliaire train_test_split de scikit-learn5 pour diviser le jeu de données Iris en sous-ensembles d'entraînement et de test, puis entraînez le modèle.

(b) Effectuez maintenant une évaluation afin de calculer la précision de votre modèle à l'aide du score d'exactitude accuracy_score de scikit-learn6.

(c) En classification binaire, nous évaluons intuitivement le modèle à l'aide des métriques de précision et de rappel. Toutefois, la classification multiclasse est différente. Dans ce cas, le package scikit-learn fournit l'implémentation des scores de précision et de rappel fondés sur les moyennes macro et micro : « micro » calcule les métriques globalement en comptant le nombre total de vrais positifs, de faux négatifs et de faux positifs. La version « macro » calcule les métriques pour chaque étiquette et détermine leur moyenne non pondérée.

Effectuez une évaluation de vos résultats et calculez les scores de précision et de rappel avec les moyennes micro et macro, à l'aide de precision_score7 et recall_score8 de scikit-learn.

Livrable :

1) Créez un fichier PDF contenant les réponses aux questions théoriques et les questions d'implémentation. Vous devez nommer votre fichier selon la convention suivante : A#_studentID, où # est le numéro du devoir. studentID est votre numéro d'étudiant.

2) Les devoirs doivent être remis dans la section des devoirs (Léa) au plus tard le 1 – Nov. – 2026 à 23:55.

[4] https://scikit-learn.org/stable/auto_examples/datasets/plot_iris_dataset.html

[5] https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

[6] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

[7] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html

[8] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html